

Project FORESIGHT — AI-Powered Demand & Inventory Intelligence Platform

Client engagement: NorthBay Living (D2C home & lifestyle) · Intern: Eswar Mahalingam · Track: Data Science & Analytics · Duration: 4 weeks · Submitted: September 2026

Live deployment: <https://foresight-northbay.netlify.app> · Scoring API: <https://foresight-northbay.netlify.app/api/score?sku=SKU012> · Source: GitHub repository [foresight](#) (README has setup and results)

1. Overview

Project FORESIGHT is an AI-powered demand forecasting and inventory intelligence platform built for NorthBay Living, a mid-size direct-to-consumer home & lifestyle brand that plans stock on spreadsheets and gut feel. The client loses money in two directions at once: best-sellers stock out (lost sales) and slow movers pile up (cash locked, later marked down). The engagement brief asked for four things — a weekly SKU-level demand forecast that measurably beats a naive baseline, a stockout early-warning, an overstock flag, and an interface a non-technical operations team can use unaided — plus a deployed scoring service and an executive readout.

The delivered platform ingests four raw extracts (daily sales, SKU master, calendar, monthly inventory snapshots), cleans them in a fully coded and audited pipeline, trains a LightGBM demand model validated by rolling-origin backtesting against a seasonal-naive baseline, converts the forecast plus stock position into transparent stockout/overstock risk scores with a recommended action and a rupee value for every SKU, and serves everything through a live planning dashboard and a public scoring API. Every number in this report regenerates from the raw files with one command.

Forecast accuracy
(WAPE, backtest)

8.8 %

vs 11.2 % seasonal-naive → 21.5 % better

Validation

6 folds × 8 wk

wins every fold, 49/50 SKUs; Dec-25 holdout 9.3 %

Sales at risk identified

₹47.9 L

8 SKUs to reorder now

Capital locked identified

₹1.48 Cr

8 markdown candidates

Data processed

510,082

units · 50 SKUs · 104 weeks · ₹308.56 Cr

2. Objectives and success metrics (from the brief)

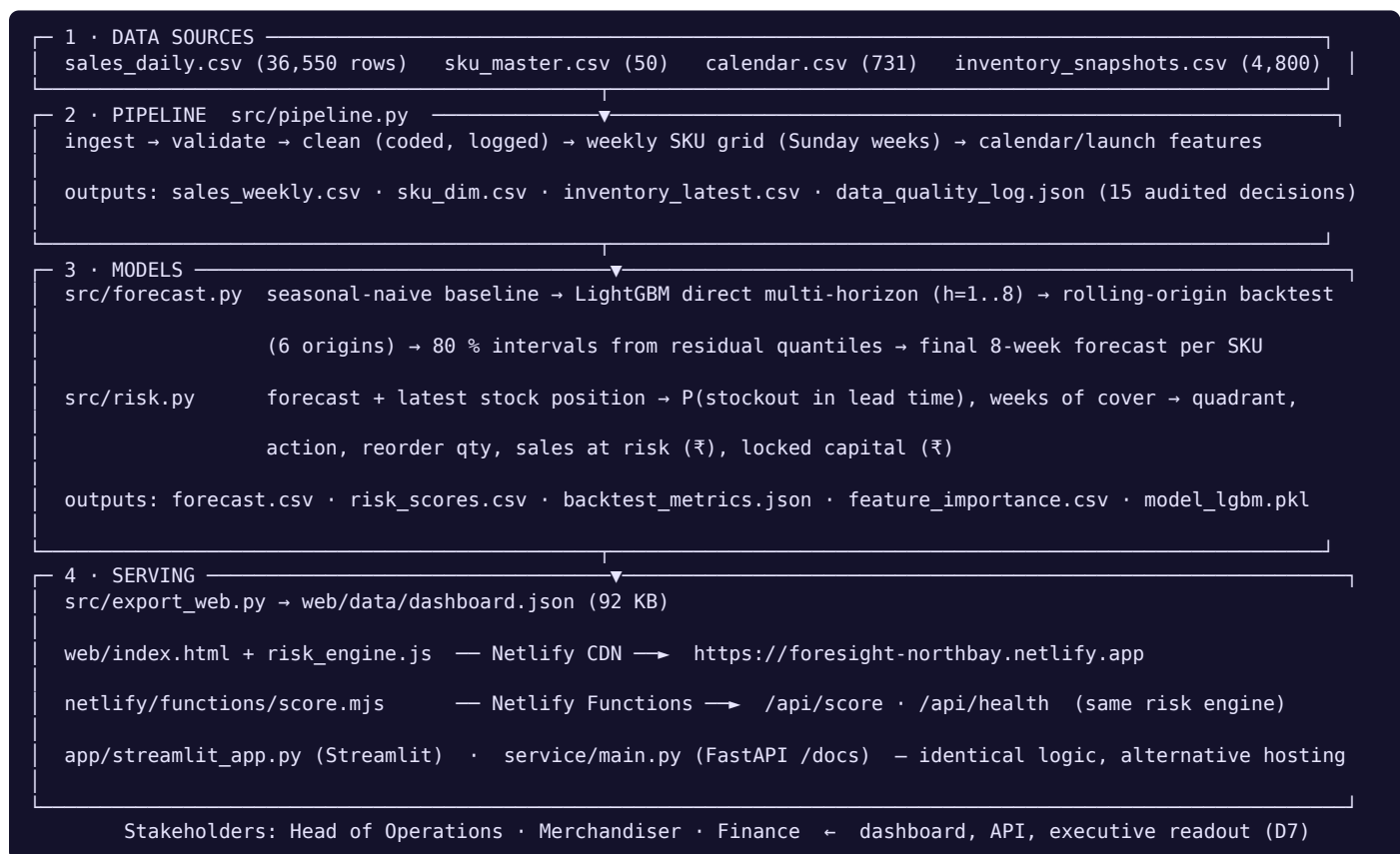
Objective	Success metric	Achieved
Weekly SKU-level forecast over a defined horizon beating a naive baseline	WAPE beats seasonal-naive by a clear margin on backtest	8-week horizon; WAPE 8.8 % vs 11.2 % (22 % better); bias +0.3 %
Flag stockout and overstock risk for every SKU with a clear action	Flags right often enough to act on	All 50 SKUs scored; 4-quadrant grid; probability-based, transparent formulas; 80 % interval calibrated at 80 % on holdout
Quantify the business impact in rupees	Rupee figure for sales-at-risk and locked capital	₹47.9 L sales at risk; ₹1.48 Cr locked capital; per SKU and per category
Dashboard the ops team can use unaided + deployed scoring service	Usability; public URL	Live dashboard and API at https://foresight-northbay.netlify.app ; filters, lists, what-if, CSV export, loading/empty/error states
Reproducibility	Pipeline re-runs end-to-end from raw data	<code>python src/run_all.py</code> (~1 min); fixed seed; 5 automated tests incl. leakage guard

3. Tech stack

Layer	Technology	Why
Language & data	Python 3.11, pandas, NumPy	Standard analytics stack; readable, auditable transformations
Modelling	LightGBM (Tweedie objective), scikit-learn metrics, SciPy (normal CDF for risk)	Gradient-boosted trees handle 50 heterogeneous series with shared seasonality/promo effects in one model; Tweedie suits non-negative count-like demand
Visualisation	matplotlib (report figures), Plotly (Streamlit), Chart.js 4 (web dashboard)	Publication-quality static charts; interactive hover/filter for the ops team
Dashboard	Streamlit app (app/) and a static HTML/JS dashboard (web/)	Streamlit for fast local iteration as recommended; static site for a zero-cold-start public deployment on Netlify
Scoring service	FastAPI + Pydantic (service/) and a Netlify serverless function (netlify/functions/score.mjs) sharing one risk engine	Documented inputs/outputs, validation, graceful errors; the serverless twin gives a free, always-on public endpoint
Deployment	Netlify (static + functions), deployable to Streamlit Community Cloud / Render	Public URL requirement; CDN-served, HTTPS
Quality	pytest, executed Jupyter notebooks, Git	Regression tests for metrics, risk logic, leakage and API errors; notebooks show the analysis with outputs

4. Architecture

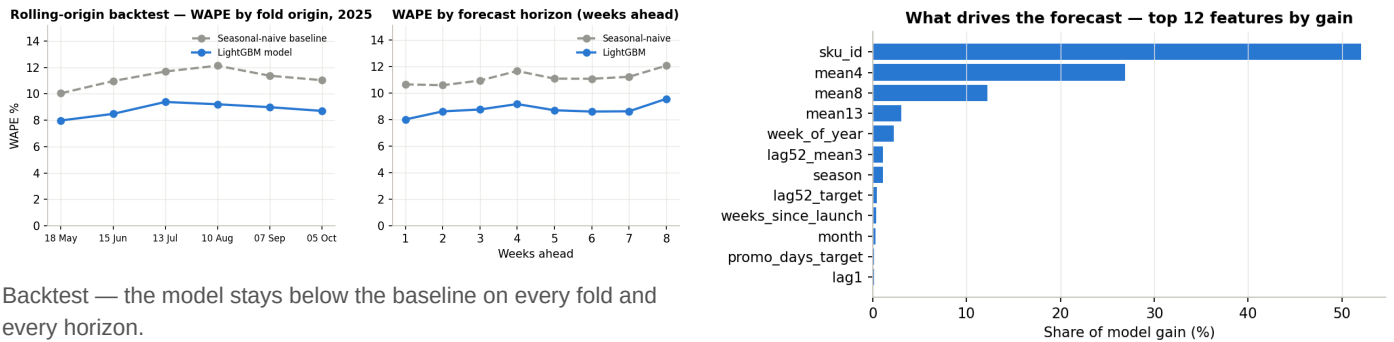
FORESIGHT keeps four layers separate so the client can re-run and trust each one: data sources → reproducible pipeline → models → serving. Raw extracts never leave `data/raw/`; every downstream artefact is regenerated.



Modelling workflow — baseline first

- (1) **Frame:** 8-week horizon, WAPE primary, bias secondary; forecast origin 2025-11-30 chosen so the latest inventory snapshot (1-Dec-2025) and the forecast start coincide, leaving four December weeks as a true holdout.
- (2) **Baseline:** seasonal-naive, $\text{forecast}(t+h) = \text{actual}(t+h-52)$.
- (3) **Features:** lags 1–13, rolling mean/std, trend, last-

year same week ± 1 , this-year/last-year level ratio, planned promo and holiday days of the target week, week-of-year, month, season, weeks since launch, price, SKU and category identity, horizon h — all computed strictly from weeks $\leq$ origin. (4) **Model**: one LightGBM regressor over rows (SKU $\times$ origin $\times$ h). (5) **Backtest**: six rolling origins in 2025, four weeks apart, retrained from scratch each time. (6) **Evaluate**: compare to baseline per fold, horizon, category and SKU. (7) **Risk**: combine with inventory position.

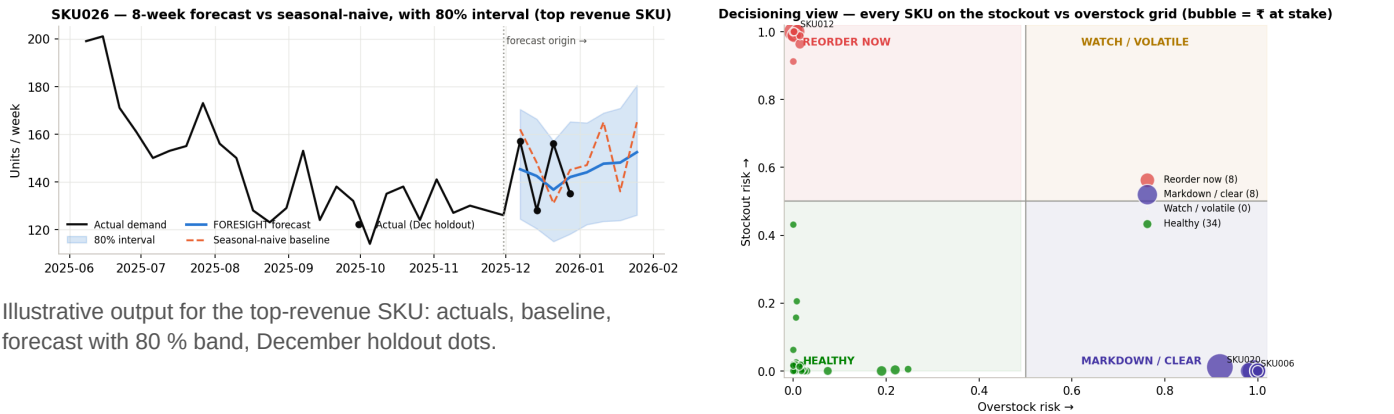


Backtest — the model stays below the baseline on every fold and every horizon.

Drivers — product identity and recent level dominate; last-year same-week and season carry the seasonality.

5. Results

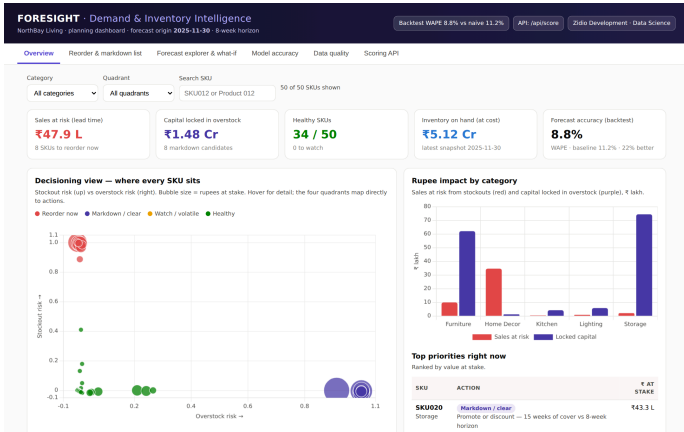
Fold origin			Weeks ahead			Category		
Model	Naive		Model	Naive		Model	Naive	
2025-05-18	8.0 %	10.0 %	1.0	8.0 %	10.7 %	Furniture	9.7 %	12.1 %
2025-06-15	8.5 %	11.0 %	2.0	8.6 %	10.6 %	Home Decor	8.3 %	10.4 %
2025-07-13	9.4 %	11.7 %	3.0	8.8 %	10.9 %	Kitchen	8.8 %	11.5 %
2025-08-10	9.2 %	12.1 %	4.0	9.2 %	11.7 %	Lighting	8.4 %	10.8 %
2025-09-07	9.0 %	11.4 %	5.0	8.7 %	11.1 %	Storage	9.0 %	11.5 %
2025-10-05	8.7 %	11.0 %	6.0	8.6 %	11.1 %	Overall: WAPE 8.8 % vs 11.2 %; MAPE 10.8 % vs 13.8 %; bias +0.32 %. Dec-2025 holdout: 9.3 % vs 12.4 %; 80 % interval coverage 79.5 %.		
			7.0	8.6 %	11.2 %			
			8.0	9.6 %	12.1 %			



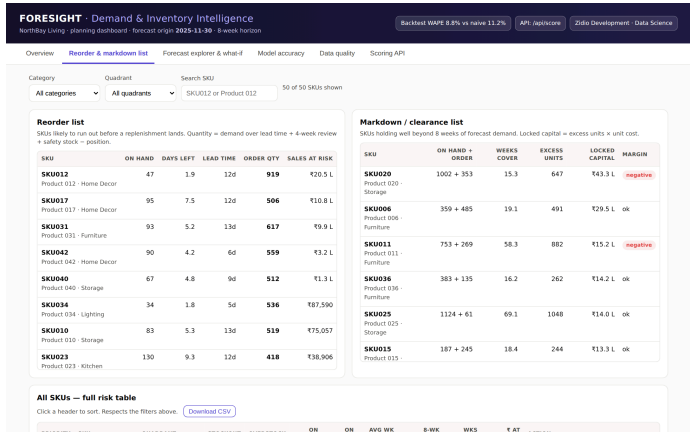
Illustrative output for the top-revenue SKU: actuals, baseline, forecast with 80 % band, December holdout dots.

Decisioning grid: 8 reorder, 8 markdown, 34 healthy.

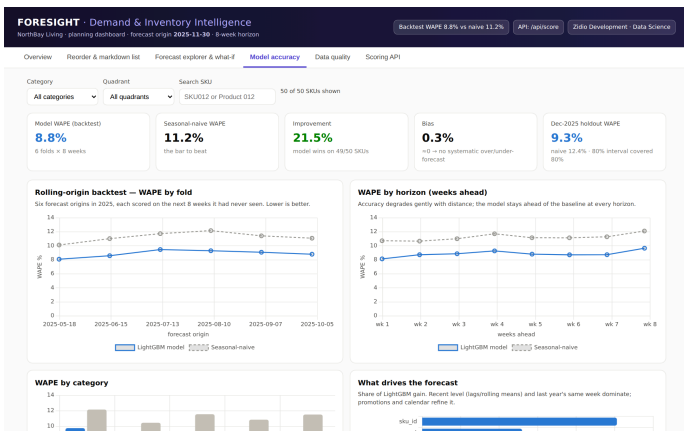
6. Screenshots — live deployment



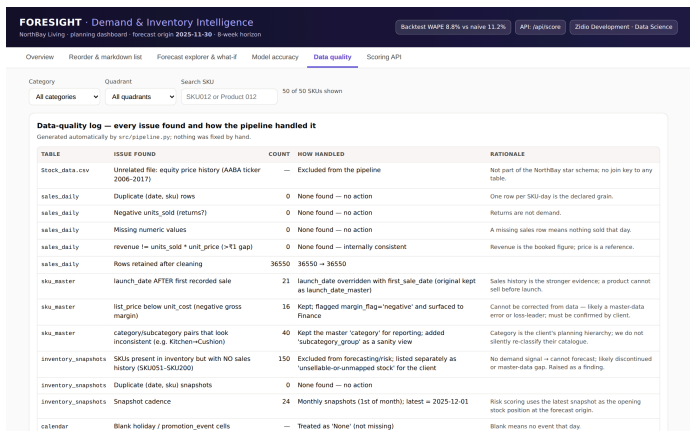
Overview — KPIs, decisioning grid, rupee impact by category, top priorities, total demand with forecast band.



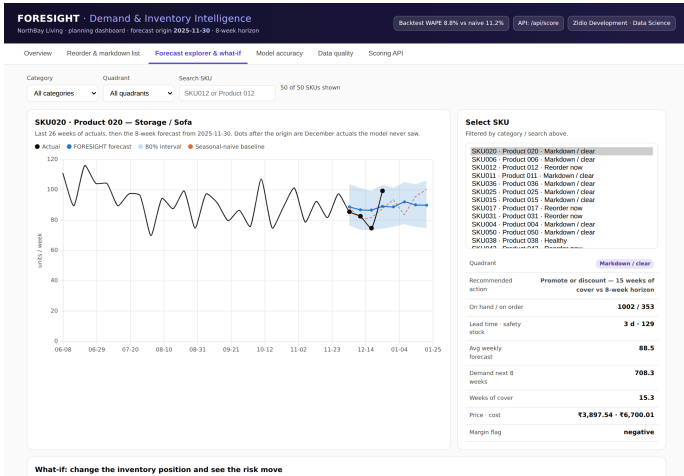
Reorder & markdown lists with order quantities, plus a sortable full table and CSV export.



Model accuracy — backtest by fold/horizon/category, feature importance and plain-language limits.



Data-quality log rendered straight from the pipeline output.



Forecast explorer — per-SKU history, forecast, 80 % band, seasonal-naive and holdout actuals; what-if controls re-score risk instantly with the same formula as the batch job.

```
GET https://foresight-northbay.netlify.app/api/score?sku=SKU012&lead_time_days=21 → HTTP 200

{
  "forecast_origin": "2025-11-30",
  "count": 1,
  "results": [
    {
      "sku_id": "SKU012",
      "product name": "Product 012",
      "category": "Home Decor",
      "subcategory": "Table",
      "inventory position": {
        "on_hand": 47,
        "on_order": 60,
        "lead_time_days": 21,
        "safety_stock": 16,
        "overridden": true
      },
      "forecast": {
        "origin": "2025-11-30",
        "horizon weeks": 8,
        "weeks": [
          "2025-12-07",
          "2025-12-14",
          "2025-12-21",
          "2025-12-28",
          "2026-01-04",
          "2026-01-11",
          "2026-01-18",
          "2026-01-25"
        ],
        "mean": [
          162.6,
          165.8,
          163.9,
          166.7,
          171.9,
          178.5,
          181.7,
          180.6
        ],
        "lo80": [
          139.2,
          140.2,
          137.9,
          138.7,
          145.8,
          149.3,
          149.3,
          149.3
        ]
      }
    }
  ]
}
```

Live scoring API response (what-if override `lead_time_days=21` on SKU012). Invalid input returns explicit JSON errors (400/404/405), never a crash.

7. Data quality and key assumptions

Issue found	n	Handling
[Stock_data.csv] Unrelated file: equity price history (AABA ticker 2006–2017)	—	Excluded from the pipeline
[sales_daily] Duplicate (date, sku) rows	0	None found — no action
[sales_daily] Negative units_sold (returns?)	0	None found — no action
[sales_daily] Missing numeric values	0	None found — no action
[sales_daily] revenue != units_sold * unit_price (>₹1 gap)	0	None found — internally consistent
[sales_daily] Rows retained after cleaning	36550	36550 → 36550
[sku_master] launch_date AFTER first recorded sale	21	launch_date overridden with first_sale_date (original kept as launch_date_master)
[sku_master] list_price below unit_cost (negative gross margin)	16	Kept; flagged margin_flag='negative' and surfaced to Finance
[sku_master] category/subcategory pairs that look inconsistent (e.g. Kitchen → Cushion)	40	Kept the master 'category' for reporting; added 'subcategory_group' as a sanity view
[inventory_snapshots] SKUs present in inventory but with NO sales history (SKU051–SKU200)	150	Excluded from forecasting/risk; listed separately as 'unsellable-or-unmapped stock' for the client
[inventory_snapshots] Duplicate (date, sku) snapshots	0	None found — no action
[inventory_snapshots] Snapshot cadence	24	Monthly snapshots (1st of month); latest = 2025-12-01
[calendar] Blank holiday / promotion_event cells	—	Treated as 'None' (not missing)
[sales_weekly] Partial weeks at series edges (fewer than 7 days)	50	Dropped
[sales_weekly] SKU-weeks with no sales rows at all	0	Filled with 0 units

Assumptions: the latest monthly snapshot is the opening stock position; on-order stock arrives at the end of the lead time; promotions/holidays beyond the calendar file are absent; lead-time demand is approximately normal with sigma scaled from backtest relative errors; two years of history means one prior seasonal cycle. **Out of scope by design:** price optimisation, supplier selection, live integrations, automated purchase orders.

8. Deliverables map (brief D1–D7 → Zidio submission items)

Brief	Deliverable	Where
D1	Reproducible data pipeline + audited data-quality log	src/pipeline.py, data/processed/, notebook 01
D2	Data-quality & EDA insight memo	reports/D2_EDA_Data_Quality_Memo.pdf, reports/figures/
D3	Demand forecast model, backtested vs seasonal-naive	src/forecast.py, outputs/backtest_metrics.json, notebooks 02–03
D4	Risk scoring with actions and rupee impact	src/risk.py, outputs/risk_scores.csv
D5	Planning dashboard	https://foresight-northbay.netlify.app (static) · app/streamlit_app.py
D6	Deployed scoring service	https://foresight-northbay.netlify.app/api/score · service/main.py (FastAPI)
D7	Executive readout	reports/D7_Executive_Readout.pdf (9 slides)
Zidio 1–5	Source code · Live deployment · Demo video · Feedback video · Project report	GitHub repo · Netlify URL · scripts in reports/Video_Scripts_Demo_and_Feedback.pdf · this document

9. Challenges and learnings

- **Aligning time.** Inventory is a monthly snapshot while sales are daily; choosing the forecast origin to coincide with the snapshot date (rather than the last data point) made the risk scores meaningful and created a free holdout month.

- **Leakage discipline.** Building a direct multi-horizon design where every feature is computed from history $\leq$ origin — and writing a test that asserts it — was more work than the model itself, and is the reason the backtest can be trusted.
- **Calibrating the risk rule.** A first version flagged 36 of 50 SKUs for reorder because it compared 8-week demand with stock on hand; reframing stockout as "will it run out before the next order can land" (lead-time demand vs on-hand, reorder-point check) produced a triage the ops team can act on: 8 / 8 / 34.
- **Two dashboards, one truth.** Streamlit is fastest to build, but a static page with a serverless function gave a zero-cold-start public URL. Sharing a single risk engine (Python and a line-for-line JavaScript port) kept both consistent.
- **Catalogue problems are business findings.** Below-cost prices and 150 inventory SKUs with no sales are not "data cleaning" — they went into the executive readout as decisions for Finance.

10. Conclusion

FORESIGHT meets every acceptance criterion in the brief with evidence: an honestly validated forecast (8.8 % WAPE, 22 % better than seasonal-naïve across 6 rolling folds and a true holdout), a transparent risk layer that converts it into two short action lists worth ₹47.9 L of protected sales and ₹1.48 Cr of releasable cash, and a live dashboard plus API that NorthBay's operations team can use monthly without a data scientist. Next steps recommended to the client: adopt the monthly refresh, share the promotion calendar eight weeks ahead, fix the master-data issues surfaced here, and monitor forecast error against the baseline so the system earns trust every month it runs.